

IMT MINES ALÈS
Gestion du cycle de vie des certificats

TP

Cryptographie

Construction et exploitation d'une infrastructure de gestion de clés

Table des matières

1	Introduction	2
1.1	Objectifs	2
1.2	Choix de dimensionnement (recommandations ANSSI)	2
1.3	Durées de validité demandées	2
2	Partie 1 : Étude de certificats existants	3
2.1	Récupération du certificat (Q1)	3
2.2	Lecture de la chaîne (Q2 à Q5)	3
2.3	Analyse du certificat (Q6 à Q16)	3
2.4	La révocation (Q17 à Q21)	5
3	Partie 2 : Construction d'une PKI multi-autorités	7
3.1	La certification croisée n'est que dans un sens	7
3.2	Tâche 1 : Méthodologie de mise en place	7
3.2.1	Arborescence dédiée et fichiers de base de données	7
3.2.2	Fichier de configuration OpenSSL (un par AC)	7
3.2.3	Création de la racine RSA (auto-signée)	8
3.2.4	Création de la racine ECDSA (auto-signée)	8
3.2.5	Création des autorités subordonnées	9
3.3	Révocation par CRL (tâches 2 à 4)	10
3.4	Signature et chiffrement S/MIME (tâches 5 à 7)	10
3.5	Révocation par OCSP (tâches 8 à 12)	11
3.6	Certification croisée (tâches 13 et 14)	12
3.7	Magasin de certificats Windows (tâche 15)	12
3.7.1	Pourquoi un « magasin » de certificats	13
3.7.2	Certificats à importer	13
3.7.3	Import dans les magasins (PowerShell administrateur)	13
3.7.4	Vérification de la chaîne	13
4	Partie 3 : SoftHSM2 et ADCS	15
4.1	Autorité racine Linux avec SoftHSM2	15
4.1.1	Initialisation du token SoftHSM2	15
4.1.2	Clé RSA 4096 dans le token et racine auto-signée	15
4.2	CRL / ARL de la racine HSM	16
4.3	Absence d'export de clé privée	16
4.4	Point technique PKCS#11	18
4.5	Autorité subordonnée ADCS Windows	18
4.5.1	Objectif et vocabulaire	18
4.5.2	Premier passage Windows : rôle ADCS et génération de la CSR	19
4.5.3	Signature de la CSR côté Linux (racine SoftHSM2)	19
4.5.4	Second passage Windows : import de la racine et finalisation	20
4.5.5	Vérification de la chaîne hétérogène Linux → Windows	21
4.5.6	Émission d'un certificat TLS par l'AC ADCS	21

1 Introduction

1.1 Objectifs

Ce TP se décompose en trois parties :

- **Partie 1 : Étude de certificats existants.** Récupération et analyse du certificat TLS du site <https://cyber.gouv.fr/>, lecture de la chaîne de certification, des extensions, de la CRL, et vérification de statut de révocation par CRL puis par OCSP.
- **Partie 2 : Construction d'une PKI multi-autorités.** Mise en place d'une hiérarchie inspirée de Let's Encrypt (deux racines, RSA et ECDSA, deux autorités subordonnées), avec révocation par CRL et par OCSP, échange S/MIME et *certification croisée* entre les deux racines.
- **Partie 3 : Racine adossée à un HSM.** Création d'une autorité racine dont la clé privée ne quitte jamais un module cryptographique (HSM virtuel SoftHSM2, via PKCS#11), puis signature d'une autorité subordonnée Microsoft ADCS.

1.2 Choix de dimensionnement (recommandations ANSSI)

Entité	Algorithme	Taille / courbe	Hachage
RSA_Root_CA_APV (racine)	RSA	4096 bits	SHA-256
Sub_RSA_CA_1_APV (sous-AC)	RSA	3072 bits	SHA-256
EC_Root_CA_APV (racine)	ECDSA	secp384r1 (P-384)	SHA-256
Sub_EC_CA_1_APV (sous-AC)	ECDSA	secp384r1 (P-384)	SHA-256
Entité finale RSA	RSA	3072 bits	SHA-256
Entité finale EC	ECDSA	prime256v1 (P-256)	SHA-256
Racine HSM (Partie 3)	RSA	4096 bits	SHA-256

1.3 Durées de validité demandées

Objet	Durée	Valeur -days / CRL
AC racines	15 ans	5479 jours
AC subordonnées	10 ans	3653 jours
Entités finales	1 an	365 jours
CRL / ARL	1 mois	default_crl_days = 30
Racine HSM (Partie 3)	10 ans	3653 jours
AC subordonnée ADCS	5 ans	1825 jours

2 Partie 1 : Étude de certificats existants

2.1 Récupération du certificat (Q1)

Q1. Quel protocole sécurise la connexion dans un navigateur ?

Réponse

La connexion HTTPS est sécurisée par TLS. La connexion OpenSSL observée a utilisé TLSv1.3. SSL est l'ancien nom historique, aujourd'hui, le protocole utilisé est TLS.

```
openssl s_client -connect cyber.gouv.fr:443 -servername cyber.gouv.fr -showcerts \
2>/dev/null </dev/null > cyber.gouv.fr.chain.txt
awk '/BEGIN CERTIFICATE/{c++} END{print c}' cyber.gouv.fr.chain.txt
openssl s_client -connect cyber.gouv.fr:443 -servername cyber.gouv.fr 2>/dev/null </dev/null \
| openssl x509 -out cyber.gouv.fr.pem
```

2

2.2 Lecture de la chaîne (Q2 à Q5)

Réponse

Q2. Longueur de la chaîne : la chaîne fournie par `openssl s_client` contient 2 certificats : le certificat serveur `cyber.gouv.fr` et le certificat subordonné `GandiCert`. La racine n'est pas envoyée par le serveur car elle est supposée présente dans le magasin de confiance du client.

Q3. AC racine : DigiCert Global Root G2. Elle n'est pas fournie dans la chaîne TLS récupérée, mais constitue l'ancrage de confiance de la chaîne `GandiCert`.

Q4. AC subordonnée : le certificat serveur est émis par C=FR, O=Gandi SAS, CN=GandiCert.

Q5. Taille de la clé publique du site : le certificat `cyber.gouv.fr` contient une clé publique RSA 4096 bits.

2.3 Analyse du certificat (Q6 à Q16)

```
openssl x509 -in cyber.gouv.fr.pem -noout -text > cyber.gouv.fr.x509.txt
```

Q6. Type de certificat.

Réponse

Il s'agit d'un certificat X.509 v3 d'entité finale pour serveur TLS. Le sujet contient CN=`cyber.gouv.fr`. Les SAN observés sont DNS:`cyber.gouv.fr`, DNS:`ssi.gouv.fr`, DNS:`www.ssi.gouv.fr` et DNS:`www.cyber.gouv.fr`. L'Extended Key Usage contient TLS Web Server Authentication et les Basic Constraints indiquent CA:FALSE.

Q7. Peut-il signer d'autres certificats ?

Réponse

Non. Le certificat contient Basic Constraints: CA:FALSE. Son Key Usage vaut Digital Signature, Key Encipherment et ne contient pas keyCertSign. Il ne peut donc pas signer d'autres certificats.

Q8. Peut-il signer du courrier électronique ?

Réponse

Non pour l'usage S/MIME. L'Extended Key Usage contient uniquement TLS Web Server Authentication et ne contient pas E-mail Protection (emailProtection).

Q9. Où trouve-t-on la CRL de l'AC émettrice ?**Réponse**

Les points de distribution CRL réellement présents sont :

```
http://crl3.digicert.com/GandiCert.crl
http://crl4.digicert.com/GandiCert.crl
```

Q10. Format du numéro de série et conversion décimale.**Réponse**

Le numéro de série est affiché en hexadécimal par OpenSSL.

```
Serial hex avec separateurs : 05:7e:1f:dd:c4:8e:7a:07:80:64:3a:be:eb:cc:52:a5
Serial hex normalise       : 057E1FDDC48E7A0780643ABEEBCC52A5
Serial decimal              : 7301015708052902158374241928634716837
```

Il s'agit d'un entier ASN.1 affiché en hexadécimal, la valeur décimale montre qu'il s'agit d'un grand nombre non trivial à prédire.

Q11. Afficher la clé publique.

```
openssl x509 -in cyber.gouv.fr.pem -pubkey -noout
```

```
Public Key Algorithm: rsaEncryption
Public-Key: (4096 bit)
Exponent: 65537 (0x10001)
```

Q12. Afficher la clé privée.**Réponse**

C'est impossible à partir du certificat public. Un certificat X.509 contient la clé publique, l'identité, les extensions et la signature de l'autorité, mais jamais la clé privée. Celle-ci reste uniquement côté serveur.

Q13 à Q16. Lecture et vérification de la CRL.

```
wget -O GandiCert.crl http://crl3.digicert.com/GandiCert.crl
openssl crl -inform DER -in GandiCert.crl -text -noout > crl.txt
rg "057E1FDDC48E7A0780643ABEEBCC52A5|05:7E:1F:DD:C4:8E:7A:07:80:64:3A:BE:EB:CC:52:A5" crl.txt
```

Réponse

Q13. La CRL consultée indique :

```
Last Update: Jun 22 12:45:56 2026 GMT
Next Update: Jun 29 12:45:56 2026 GMT
```

Sa durée de validité est donc de 7 jours.

Q14. Les premières entrées affichées dans `crl.txt` ne montrent que le numéro de série et la date de révocation. Des champs `X509v3 CRL Reason Code` apparaissent plus loin, mais pas systématiquement pour les premières entrées.

Q15. Pour vérifier rapidement que la CRL est la bonne, on contrôle que son **Issuer** correspond à l'AC émettrice du certificat (`C=FR, O=Gandi SAS, CN=GandiCert`), que l'URL provient bien des `CRL Distribution Points`, puis que la signature de la CRL est vérifiable avec le certificat de cette AC.

Q16. La recherche du numéro de série du certificat `cyber.gouv.fr` dans la CRL ne renvoie aucun résultat : le certificat n'apparaît donc pas comme révoqué dans la CRL consultée.

2.4 La révocation (Q17 à Q21)

Site `https://revoked-rsa-dv.ssl.com/` dans 3 navigateurs.

SSL.com - Test Website

This is a test website authenticated by [SSL.com](https://www.ssl.com).

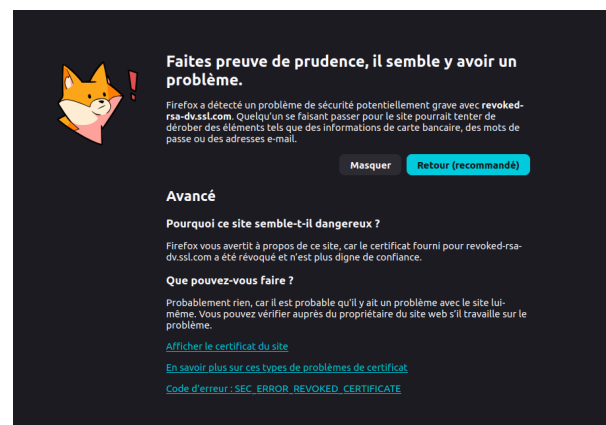


Figure 1: Pages d'erreur de sécurité pour `https://revoked-rsa-dv.ssl.com` : à gauche Chrome, à droite Firefox.

Q17. Le série du certificat révoqué est-il dans la CRL ?

Réponse

Oui. Le certificat testé est `01-public-certs/revoked-rsa-dv.ssl.com.pem`. Son numéro de série est :

```
1811B09C4BB9E179133BB9D9A9B140C5
```

La CRL utilisée est `http://crls.ssl.com/SSL.com-TLS-I-RSA-R1.crl`. Le numéro y apparaît avec la date de révocation suivante :

```
Serial Number: 1811B09C4BB9E179133BB9D9A9B140C5
Revocation Date: Jun 9 14:37:38 2026 GMT
```

Q18. Vérification OCSP.

```
openssl ocsp \
  -issuer ssl.com-tls-i-rsa-r1.pem \
  -cert revoked-rsa-dv.ssl.com.pem \
  -url http://ocspssl.com \
  -no_nonce \
  -resp_text -text
```

```
OCSP Response Status: successful (0x0)
Cert Status: revoked
Revocation Time: Jun  9 14:37:38 2026 GMT
revoked-rsa-dv.ssl.com.pem: revoked
```

Réponse

La CRL et OCSP concordent : le certificat est bien révoqué.

Q19. Numéros de série de google.com et youtube.com.

```
google.com : 139CDF29A8B5BC5212413EEEEAFD18CBE
youtube.com : 139CDF29A8B5BC5212413EEEEAFD18CBE
```

Réponse

Les deux hôtes ont présenté le même certificat feuille, émis par C=US, O=Google Trust Services, CN=WR2, avec CN=*.google.com. Les SAN contiennent à la fois DNS:google.com et DNS:youtube.com. Il s'agit donc d'un certificat multi-SAN partagé par l'infrastructure Google.

Q20. https://wrong.host.badssl.com/.

```
verify error:num=62:hostname mismatch
Verification error: hostname mismatch
```

Réponse

L'alerte est cohérente : le nom demandé `wrong.host.badssl.com` ne correspond pas au certificat présenté. La validation du nom d'hôte repose sur les SAN, ou à défaut sur le CN selon la logique de la RFC 6125. Un wildcard ne couvre qu'un seul niveau de sous-domaine.

Q21. Certificats racines ISRG.

Réponse

Non réalisé car on a pas les fichiers `isrg-1.crt` et `isrg-2.crt`. La méthode de lecture reste :

```
for f in isrg-1.crt isrg-2.crt; do
  openssl x509 -in $f -noout -subject -issuer -dates
  openssl x509 -in $f -noout -text | grep -E "Public Key Algorithm|Public-Key|Signature"
  ↪ Algorithm"
done
```

3 Partie 2 : Construction d'une PKI multi-autorités

3.1 La certification croisée n'est que dans un sens

Réponse

Une certification croisée consiste à faire signer la clé publique d'une racine par l'autre racine, ce qui produit un certificat supplémentaire pour la même autorité. Ici, `RSA_Root_CA_APV` signe un certificat croisé pour `EC_Root_CA_APV`. Un vérificateur qui fait confiance à la racine RSA peut donc valider une chaîne de la branche EC via ce certificat croisé. L'inverse n'est pas vrai sans certificat miroir `RSA_Root_CA_APV` signé par `EC_Root_CA_APV`.

Réponse

Exemple de chaîne croisée unidirectionnelle :

```
RSA_Root_CA_APV
-> EC_Root_CA_APV (certificat croise signe par RSA_Root_CA_APV)
-> Sub_EC_CA_1_APV
-> Leaf_EC_1_APV.crosspath
```

Pour une certification croisée dans les deux sens, il faudrait émettre un second certificat croisé, cette fois avec la racine EC comme émetteur et la clé publique de la racine RSA comme sujet :

```
RSA_Root_CA_APV -> EC_Root_CA_APV
EC_Root_CA_APV -> RSA_Root_CA_APV
```

3.2 Tâche 1 : Méthodologie de mise en place

3.2.1 Arborescence dédiée et fichiers de base de données

Chaque AC dispose de son répertoire, avec `private/` (clé privée, `chmod 700`) et `db/` (base de données OpenSSL).

```
mkdir -p ca/{certs,crl,newcerts,csr,smime,ocsp}
for CA in RSA_Root_CA EC_Root_CA Sub_RSA_CA_1 Sub_EC_CA_1; do
  mkdir -p ca/$CA/db ca/$CA/private
  chmod 700 ca/$CA/private
  : > ca/$CA/db/$CA.db          # base vide mais existante
  : > ca/$CA/db/$CA.db.attr
  echo 01 > ca/$CA/db/$CA.srl   # numero de serie initial
  echo 01 > ca/$CA/db/$CA.crl.srl # numero de CRL initial
done
```

3.2.2 Fichier de configuration OpenSSL (un par AC)

Extrait de la configuration de la racine RSA (`RSA_Root_CA.conf`) :

```
[ca]
default_ca = root_ca

[root_ca]
certificate      = $dir/RSA_Root_CA.crt
private_key      = $dir/private/RSA_Root_CA.key
database         = $dir/db/RSA_Root_CA.db
serial           = $dir/db/RSA_Root_CA.srl
crlnumber        = $dir/db/RSA_Root_CA.crl.srl
default_md       = sha256          # hachage SHA-256 (ANSSI)
```



```

default_crl_days = 30          # CRL valable 1 mois
policy            = policy_loose

# --- profil de la racine elle-meme (auto-signature) ---
[root_ca_ext]
basicConstraints      = critical,CA:TRUE,pathlen:2    # 2 niveaux de sous-AC
keyUsage              = critical,keyCertSign,cRLSign
subjectKeyIdentifier  = hash
authorityKeyIdentifier = keyid:always

# --- profil applique aux AC subordonnees signees par la racine ---
[sub_ca_ext]
basicConstraints      = critical,CA:TRUE,pathlen:0    # ne signe que des EE
keyUsage              = critical,keyCertSign,cRLSign
crlDistributionPoints = URI:http://crl.pki-apv.local/RSA_Root_CA.crl
authorityInfoAccess   = caIssuers;URI:http://aia.pki-apv.local/RSA_Root_CA.crt

```

3.2.3 Création de la racine RSA (auto-signée)

La requête est créée *et* la bi-clé RSA générée en une commande, puis la racine s'auto-signe :

```

# 1) genere la bi-cle RSA 4096 + la CSR
openssl req -new -config ca/RSA_Root_CA/RSA_Root_CA.conf \
  -newkey rsa:4096 -keyout ca/RSA_Root_CA/private/RSA_Root_CA.key -nodes \
  -out ca/csr/RSA_Root_CA.csr
# 2) auto-signature (15 ans, extensions racine)
openssl ca -batch -config ca/RSA_Root_CA/RSA_Root_CA.conf -selfsign \
  -in ca/csr/RSA_Root_CA.csr -extensions root_ca_ext -days 5479 \
  -out ca/RSA_Root_CA/RSA_Root_CA.crt

```

Vérification du certificat racine obtenu :

```

$ openssl x509 -in RSA_Root_CA.crt -noout -subject -issuer -dates
subject= C = FR, O = TP IMT Mines Ales, OU = PKI APV, CN = RSA Root CA APV
issuer = C = FR, O = TP IMT Mines Ales, OU = PKI APV, CN = RSA Root CA APV
notBefore=Jun 23 06:30:04 2026 GMT
notAfter =Jun 23 06:30:04 2041 GMT          # 15 ans
    X509v3 Basic Constraints: critical
        CA:TRUE, pathlen:2
    X509v3 Key Usage: critical
        Certificate Sign, CRL Sign

```

3.2.4 Création de la racine ECDSA (auto-signée)

OpenSSL ne génère pas la clé EC implicitement via `req` : on crée d'abord la clé P-384, puis la requête, puis l'auto-signature.

```

# 1) cle EC P-384 (secp384r1)
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:secp384r1 \
  -out ca/EC_Root_CA/private/EC_Root_CA.key
# 2) CSR a partir de la cle existante
openssl req -new -config ca/EC_Root_CA/EC_Root_CA.conf \
  -key ca/EC_Root_CA/private/EC_Root_CA.key -out ca/csr/EC_Root_CA.csr
# 3) auto-signature (15 ans, pathlen:1)
openssl ca -batch -config ca/EC_Root_CA/EC_Root_CA.conf -selfsign \
  -in ca/csr/EC_Root_CA.csr -extensions root_ca_ext -days 5479 \
  -out ca/EC_Root_CA/EC_Root_CA.crt

```

```

$ openssl x509 -in EC_Root_CA.crt -noout -text | grep -E "Algorithm|NIST|pathlen|Sign"
    Signature Algorithm: ecdsa-with-SHA256
        ASN1 OID: secp384r1
        NIST CURVE: P-384
        CA:TRUE, pathlen:1
        Certificate Sign, CRL Sign

```

3.2.5 Création des autorités subordonnées

Chaque sous-AC génère sa clé et sa CSR, puis est signée par sa racine avec le profil `sub_ca_ext` (10 ans). Exemple pour la branche RSA :

```
openssl req -new -config ca/Sub_RSA_CA_1/Sub_RSA_CA_1.conf \  
-newkey rsa:3072 -keyout ca/Sub_RSA_CA_1/private/Sub_RSA_CA_1.key -nodes \  
-out ca/csr/Sub_RSA_CA_1.csr  
# signature par la racine RSA  
openssl ca -batch -config ca/RSA_Root_CA/RSA_Root_CA.conf \  
-extensions sub_ca_ext -days 3653 \  
-in ca/csr/Sub_RSA_CA_1.csr -out ca/Sub_RSA_CA_1/Sub_RSA_CA_1.crt
```

Validation des deux chaînes de subordination :

```
$ openssl verify -CAfile RSA_Root_CA.crt Sub_RSA_CA_1.crt  
Sub_RSA_CA_1.crt: OK  
$ openssl verify -CAfile EC_Root_CA.crt Sub_EC_CA_1.crt  
Sub_EC_CA_1.crt: OK
```

3.3 Révocation par CRL (tâches 2 à 4)

Tâche 2 : émettre une entité finale par Sub_RSA_CA_1_APV. Le certificat généré se trouve dans le dossier 03-crl-ocsp : Sub_RSA_CA_1_APV-leaf-crl.crt.

```
cd /home/aurelien/Documents/certificats/03-crl-ocsp
openssl ca -batch \
  -config openssl-rsa-sub-ocsp.cnf \
  -extensions v3_leaf \
  -days 365 \
  -notext \
  -in Sub_RSA_CA_1_APV-leaf-crl.csr.pem \
  -out Sub_RSA_CA_1_APV-leaf-crl.crt
```

Tâche 3 : valider face à la chaîne d'autorité.

```
openssl verify \
  -CAfile /home/aurelien/Documents/certificats/02-pki-openssl/RSA_Root_CA_APV/certs/
  ↳ RSA_Root_CA_APV.crt \
  -untrusted /home/aurelien/Documents/certificats/02-pki-openssl/Sub_RSA_CA_1_APV/certs/
  ↳ Sub_RSA_CA_1_APV.crt \
  Sub_RSA_CA_1_APV-leaf-crl.crt
```

```
/home/aurelien/Documents/certificats/03-crl-ocsp/Sub_RSA_CA_1_APV-leaf-crl.crt: OK
```

Tâche 4 : révoquer puis générer et afficher la CRL.

```
openssl ca -batch -config openssl-rsa-sub-ocsp.cnf \
  -revoke Sub_RSA_CA_1_APV-leaf-crl.crt \
  -crl_reason keyCompromise
openssl ca -config openssl-rsa-sub-ocsp.cnf \
  -gencrl -crl days 30 \
  -out ca-work/Sub_RSA_CA_1_APV/crl.pem
openssl crl -in ca-work/Sub_RSA_CA_1_APV/crl.pem -text -noout > crl-after-revoke.txt
```

```
Issuer: C=FR, O=TP Crypto Agents, OU=PKI OpenSSL, CN=Sub_RSA_CA_1_APV
Last Update: Jun 23 07:18:21 2026 GMT
Next Update: Jul 23 07:18:21 2026 GMT
Serial Number: 1000
Revocation Date: Jun 23 07:18:21 2026 GMT
X509v3 CRL Reason Code: Key Compromise
```

Le certificat apparaît bien dans la CRL, avec le motif *Key Compromise*.

3.4 Signature et chiffrement S/MIME (tâches 5 à 7)

Tâche 5 : certificat S/MIME avec les bons usages. Le certificat 04-smime/smime-user.crt est émis par Sub_RSA_CA_1_APV. Les extensions observées sont :

- Basic Constraints: CA:FALSE ;
- Key Usage: Digital Signature, Key Encipherment ;
- Extended Key Usage: E-mail Protection ;
- SAN email : smime-user@apv.local.

```
cd /home/aurelien/Documents/certificats/04-smime
openssl ca -batch \
  -config ../02-pki-openssl/Sub_RSA_CA_1_APV/openssl-rsa-sub.cnf \
  -extfile smime-openssl.cnf \
  -extensions v3_smime_signer \
```

```
-days 365 \
-notext \
-in smime-user.csr.pem \
-out smime-user.crt
```

Tâches 6 et 7 : signature, chiffrement, déchiffrement et vérification. Une démo locale a été réalisée avec un certificat destinataire de test : `recipient-test.crt`.

```
openssl smime -sign -binary -nodetach \
-in message.txt \
-signer smime-user.crt \
-inkey smime-user.key.pem \
-certfile ../02-pki-openssl/Sub_RSA_CA_1_APV/certs/Sub_RSA_CA_1_APV.crt \
| openssl smime -encrypt -aes256 \
-out signed-encrypted-message.pem \
recipient-test.crt

openssl smime -decrypt \
-in signed-encrypted-message.pem \
-recipient recipient-test.crt \
-inkey recipient-test.key.pem \
-out decrypted-signed-message.pem

openssl smime -verify \
-in decrypted-signed-message.pem \
-CAfile ../02-pki-openssl/RSA_Root_CA_APV/certs/RSA_Root_CA_APV.crt \
-out verified-message.txt
```

Réponse

Le fichier `verified-message.txt` contient le message clair attendu.

3.5 Révocation par OCSP (tâches 8 à 12)

Tâche 8 : certificat contenant URL CRL et URL OCSP. Le certificat testé se trouve dans le dossier `03-crl-ocsp` : `Sub_RSA_CA_1_APV-leaf-ocsp.crt`.

```
CRL Distribution Points:
  URI:http://crl.localhost:2560/Sub_RSA_CA_1_APV.crl
Authority Information Access:
  OCSP - URI:http://ocsp.localhost:2560
```

Tâche 9 : répondeur OCSP pour Sub_RSA_CA_1_APV. Le répondeur local est lancé sur `127.0.0.1:2560`.

```
openssl ocsp \
-index ca-work/Sub_RSA_CA_1_APV/db/index.txt \
-port 2560 \
-rsigner ../02-pki-openssl/Sub_RSA_CA_1_APV/certs/Sub_RSA_CA_1_APV.crt \
-rkey ../02-pki-openssl/Sub_RSA_CA_1_APV/private/Sub_RSA_CA_1_APV.key.pem \
-CA ../02-pki-openssl/Sub_RSA_CA_1_APV/certs/Sub_RSA_CA_1_APV.crt \
-nrequest 1
```

Tâche 10 : vérifier : statut attendu *good*.

```
openssl ocsp \
-issuer ../02-pki-openssl/Sub_RSA_CA_1_APV/certs/Sub_RSA_CA_1_APV.crt \
-cert Sub_RSA_CA_1_APV-leaf-ocsp.crt \
-url http://127.0.0.1:2560 \
-CAfile ../02-pki-openssl/RSA_Root_CA_APV/certs/RSA_Root_CA_APV.crt \
-verify_other ../02-pki-openssl/Sub_RSA_CA_1_APV/certs/Sub_RSA_CA_1_APV.crt \
-resp_text -text -no_nonce
```

```
Cert Status: good
Response verify OK
Sub_RSA_CA_1_APV-leaf-ocsp.crt: good
```

Tâches 11 et 12 : révoquer puis re-vérifier : statut attendu *revoked*.

```
openssl ca -batch -config openssl-rsa-sub-ocsp.cnf \
-revoke Sub_RSA_CA_1_APV-leaf-ocsp.crt \
-crl_reason cessationOfOperation
```

```
Cert Status: revoked
Revocation Time: Jun 23 07:18:22 2026 GMT
Response verify OK
Sub_RSA_CA_1_APV-leaf-ocsp.crt: revoked
```

Le statut passe bien de good à revoked.

3.6 Certification croisée (tâches 13 et 14)

Tâche 13 : entité finale émise par Sub_EC_CA_1_APV. Le certificat final émis pour tester les chemins est 05-cross-cert/certs/Leaf_EC_1_APV.crosspath.crt.

Mise en place du pont croisé. La racine RSA_Root_CA_APV signe un certificat croisé pour EC_Root_CA_APV. On obtient ainsi un second certificat pour la racine EC, mais émis par la racine RSA :

```
05-cross-cert/certs/EC_Root_CA_APV.cross-signed-by-RSA_Root_CA_APV.crt
```

Tâche 14 : vérifier les deux chaînes possibles.

```
# Chaîne native : Leaf_EC -> Sub_EC_CA_1 -> EC_Root_CA
openssl verify -show_chain \
-CAfile /home/aurelien/Documents/certificats/02-pki-openssl/EC_Root_CA_APV/certs/EC_Root_CA_APV
↳ .crt \
-untrusted /home/aurelien/Documents/certificats/02-pki-openssl/Sub_EC_CA_1_APV/certs/
↳ Sub_EC_CA_1_APV.crt \
/home/aurelien/Documents/certificats/05-cross-cert/certs/Leaf_EC_1_APV.crosspath.crt

# Chaîne croisee : Leaf_EC -> Sub_EC_CA_1 -> EC_Root_CA cross-signee -> RSA_Root_CA
openssl verify -show_chain \
-CAfile /home/aurelien/Documents/certificats/02-pki-openssl/RSA_Root_CA_APV/certs/
↳ RSA_Root_CA_APV.crt \
-untrusted /home/aurelien/Documents/certificats/05-cross-cert/certs/untrusted-rsa-path.pem \
/home/aurelien/Documents/certificats/05-cross-cert/certs/Leaf_EC_1_APV.crosspath.crt
```

```
/home/aurelien/Documents/certificats/05-cross-cert/certs/Leaf_EC_1_APV.crosspath.crt: OK
depth=0: CN=Leaf_EC_1_APV
depth=1: CN=Sub_EC_CA_1_APV
depth=2: CN=EC_Root_CA_APV

/home/aurelien/Documents/certificats/05-cross-cert/certs/Leaf_EC_1_APV.crosspath.crt: OK
depth=0: CN=Leaf_EC_1_APV
depth=1: CN=Sub_EC_CA_1_APV
depth=2: CN=EC_Root_CA_APV
depth=3: CN=RSA_Root_CA_APV
```

3.7 Magasin de certificats Windows (tâche 15)

3.7.1 Pourquoi un « magasin » de certificats

Sous Linux, on valide une chaîne en désignant explicitement le fichier de l'AC de confiance (`openssl verify -CAfile`). Windows fonctionne autrement : il ne lit pas des fichiers `.crt` en vrac, il s'appuie sur des *magasins de certificats* (*certificate stores*), c'est-à-dire des bases internes du système rangées par *usage*. Pour qu'un certificat émis par notre PKI soit reconnu comme valide par Windows, il faut donc *importer* nos certificats dans les bons magasins.

Deux magasins nous intéressent, et la distinction est essentielle :

- **Autorités de certification racines de confiance** (chemin PowerShell `Cert:\LocalMachine\Root`) : ce sont les *ancres de confiance*. Toute chaîne qui remonte jusqu'à une racine placée ici est jugée fiable par le système. On n'y met *que* des racines auto-signées.
- **Autorités de certification intermédiaires** (`Cert:\LocalMachine\CA`) : on y place les AC *subordonnées*. Elles ne sont pas des ancres de confiance ; elles servent à Windows pour reconstruire le chemin entre un certificat d'entité finale et la racine.

Note

`LocalMachine` désigne le magasin de *l'ordinateur* (valable pour tous les comptes et pour les services comme IIS) ; `CurrentUser` ne concernerait que la session courante. Pour une PKI de serveur, on importe toujours dans `LocalMachine`, ce qui exige une console PowerShell **en administrateur**.

3.7.2 Certificats à importer

On illustre ici avec la chaîne de la Partie 3 (racine adossée au HSM) :

- racine : `rootCA.crt` (CN=RootCA, auto-signée, à mettre dans `Root`) ;
- intermédiaire : le certificat de l'AC subordonnée ADCS `sub_ca.crt` (CN=Sub-ADCS-CA-APV, à mettre dans `CA`).

La même méthode s'applique à la chaîne OpenSSL de la Partie 2 (`RSA_Root_CA_APV.crt` en racine, `Sub_RSA_CA_1_APV.crt` en intermédiaire).

3.7.3 Import dans les magasins (PowerShell administrateur)

```
# Racine HSM -> magasin des racines de confiance de la machine
Import-Certificate -FilePath C:\TP-Crypto-ADCS\input\rootCA.crt '
-CertStoreLocation Cert:\LocalMachine\Root

# AC subordonnee ADCS -> magasin des AC intermediaires
Import-Certificate -FilePath C:\TP-Crypto-ADCS\input\sub_ca.crt '
-CertStoreLocation Cert:\LocalMachine\CA
```

Note

L'équivalent en ligne de commande historique est `certutil -addstore Root rootCA.crt` et `certutil -addstore CA sub_ca.crt`. C'est d'ailleurs ce qui a été utilisé en Partie 3 ; l'import effectif de la racine dans le magasin `Root` est visible figure 12.

3.7.4 Vérification de la chaîne

Deux contrôles complémentaires :

```
# 1) Inspection graphique : ouvrir la console des certificats machine
certlm.msc
# Double-cliquer le certificat, onglet "Chemin d'accès de certification".

# 2) Verification en ligne de commande de la chaîne
certutil -verify C:\TP-Crypto-ADCS\input\sub_ca.crt
```

Réponse

Résultat (cf. Partie 3) : dans l'onglet *Chemin d'accès de certification*, la chaîne **RootCA** → **Sub-ADCS-CA-APV** s'affiche complète avec la mention « Ce certificat est valide » (figure 13), et `certutil -verify` reconstruit la chaîne jusqu'à la racine sans erreur (figure 14). Cela confirme que Windows fait bien confiance à notre racine importée. Le même principe vaut pour un certificat feuille TLS émis ensuite par l'AC subordonnée.

4 Partie 3 : SoftHSM2 et ADCS

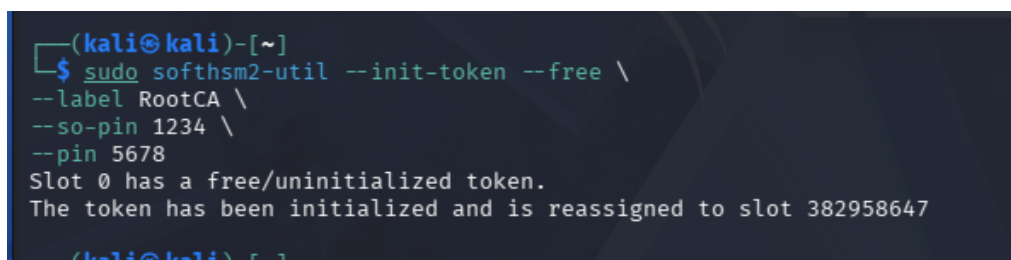
4.1 Autorité racine Linux avec SoftHSM2

La racine de cette partie a été construite sous Linux (Kali) avec **SoftHSM2**, un HSM logiciel qui expose une interface **PKCS#11**. L'intérêt : la clé privée de la racine est générée *dans* le token et n'en sort jamais ; OpenSSL ne fait que la piloter à distance via PKCS#11.

4.1.1 Initialisation du token SoftHSM2

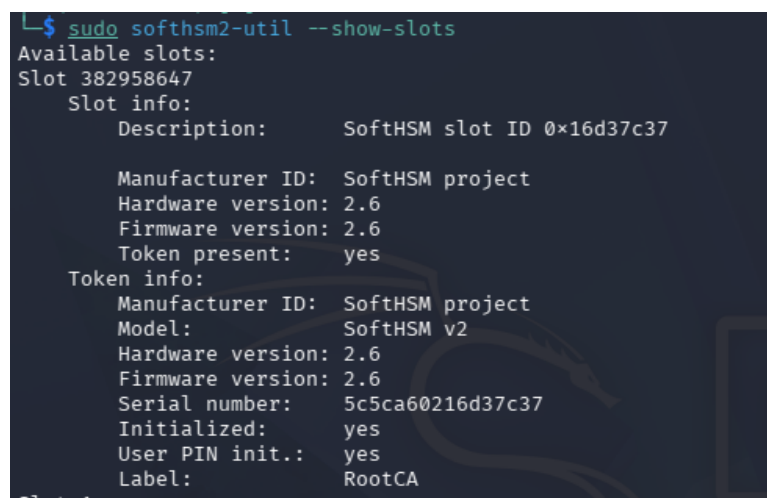
On crée un token dédié, étiqueté **RootCA**, avec un code SO (administration) et un code utilisateur (PIN) distincts :

```
softhsm2-util --init-token --free --label RootCA --so-pin 1234 --pin 5678
softhsm2-util --show-slots
```



```
(kali@kali)-[~]
$ sudo softhsm2-util --init-token --free \
--label RootCA \
--so-pin 1234 \
--pin 5678
Slot 0 has a free/uninitialized token.
The token has been initialized and is reassigned to slot 382958647
(kali@kali)-[~]
```

Figure 2: Initialisation du token SoftHSM2 RootCA : le slot libre est initialisé et réassigné.



```
$ sudo softhsm2-util --show-slots
Available slots:
Slot 382958647
Slot info:
  Description:      SoftHSM slot ID 0x16d37c37
  Manufacturer ID:  SoftHSM project
  Hardware version:  2.6
  Firmware version:  2.6
  Token present:     yes
Token info:
  Manufacturer ID:  SoftHSM project
  Model:            SoftHSM v2
  Hardware version:  2.6
  Firmware version:  2.6
  Serial number:     5c5ca60216d37c37
  Initialized:       yes
  User PIN init.:    yes
  Label:             RootCA
Slot 1
```

Figure 3: `softhsm2-util -show-slots` : le token RootCA est présent, initialisé, PIN utilisateur défini (SoftHSM v2).

4.1.2 Clé RSA 4096 dans le token et racine auto-signée

La bi-clé RSA 4096 est créée dans le token, puis OpenSSL 3 auto-signe le certificat racine en accédant à la clé privée *via le provider pkcs11prov* (la clé n'est jamais copiée sur le disque) :

```
openssl req -provider default -provider pkcs11prov \
-new -x509 -days 3650 -sha256 \
-subj "/C=FR/O=TP Crypto/CN=TP Root CA" \
-key "$ROOT_KEY_URI" \
-addext "basicConstraints=critical,CA:TRUE" \
```



```
-addext "keyUsage=critical,cRLSign,keyCertSign" \
-addext "subjectKeyIdentifier=hash" \
-addext "authorityKeyIdentifier=keyid:always" \
-out certs/rootCA.crt
```

```
Private Key Object: RSA 4096 bits
Modulus:
d1ee2bde68b979c51d11e57877b449bb2679d1a715ad5d790eadf14105c9c38
835a75cf3bb1dd8c709233a300196d8a8f0302f5fca3e212c991ba5c
c4d32bd1db25cb1157b567bffa70bd190423490de7f4ee8a6f5d3218ae15fa5
8e4924f1edff3873fd2b9588484af4c1dc3b059578ef81c3485879aa14d322c
6a41fe72d4ab6c2c7231b5449ec4d5d0b0eb0ce4daa119d2740b92fbc
67adecf775c6b7882478588e7ec80477da7c2ab6b300774f26a1e8a9830b954
2d894553c81ea0272534697864d01189e3e197a3a838e94fdddb9ab70468b
a15e24b5b5e4e4e25044dd79a86815904f5c15821f48cfd17723b098bb
135442bd73ec7c123ca9487817fc8faad288a90c6810e678bedd47dc7b9fbb6
8ef29275d6e073af643106f5d80582710634757465133a30063372c4659
6a37f045c9599191ab0583869e77046537874e4792f99eb0d1ef7087581
fc64fedf1b47b21d39e55983dc882ea789ba66d1ade1098b24de01539714d5
d7af711c64819661e12b704aeb7aee258c08457ec25aa78d97442eac9a8e4
e8730238723f8748c77238048e39c7a3e8e95cd522480a8e7315f4d
c9c684a3e64912fd07770769aeb9b618a26ae71f247f0eb32c09a8e5ca3
fb26538bf22a978be3f6fd0c81cfbbe127130548ff2c964650ab76198f615
Public exp: 65537 (0x10001)
Label: RootCAKey
ID: 1 (0x01)
Usage: decrypt, sign, signRecover, unmap
Access: sensitive, always sensitive, never extractable, local
url: pkcs11:model=SoftHSM20v2;manufacturer=SoftHSM20project;serial=5c3ca60216d37c37;token=RootCA;id=X01;object=RootCAKey;type=private
```

Figure 4: Création/auto-signature de la racine via PKCS#11 : OpenSSL retrouve bien la clé privée RootCAKey dans le token RootCA (module p11-kit-proxy.so).

Le certificat racine obtenu est bien une RSA 4096 bits, auto-signée, avec les extensions d'une AC :

```
(kali@kali)-[~/RootCA]
$ ls certs
rootCA.crt

(kali@kali)-[~/RootCA]
$ openssl x509 -in certs/rootCA.crt -text -noout
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number:
      02:09:b4:03:bd:ec:55:30:8e:df:9b:c4:13:02:4d:b2:ea:f5:d4:d6
    Signature Algorithm: sha256WithRSAEncryption
    Issuer: C=FR, O=TP Crypto, CN=TP Root CA
    Validity
      Not Before: Jun 23 08:07:17 2026 GMT
      Not After : Jun 20 08:07:17 2036 GMT
    Subject: C=FR, O=TP Crypto, CN=TP Root CA
    Subject Public Key Info:
      Public Key Algorithm: rsaEncryption
      Public-Key: (4096 bit)
      Modulus:
        00:d1:ce:e2:bd:e6:0b:97:9c:51:d1:1e:57:87:7b:
        40:9b:b2:67:9d:1a:71:5a:d3:d7:90:ea:df:14:19:
        5c:9c:38:03:5a:b7:5c:f5:bb:3d:db:8c:7d:90:23:
        54:38:d0:39:68:da:b4:fb:20:92:f6:fc:e1:3e:21:
        2c:a9:91:0a:5c:c4:d3:2b:1d:b2:5c:b1:15:7b:56:
        7b:ff:a7:0b:d1:90:42:34:9d:0e:7f:af:ee:8a:6f:
        5d:32:18:ae:15:fa:45:8e:49:24:f1:ed:ff:38:73:
        fd:7b:95:80:48:4a:f4:5c:1d:c3:0d:50:57:8e:f8:
```

Figure 5: openssl x509 -text sur certs/rootCA.crt : RSA 4096 bits, sujet et émetteur identiques (O=TP Crypto, CN=TP Root CA), validité 10 ans.

Note

La racine a été régénérée en fin de manipulation sous le *Common Name* RootCA (mêmes paramètres : O=TP Crypto, RSA 4096) ; c'est cette dernière, visible plus loin dans le magasin Windows, qui a signé l'AC subordonnée ADCS. Les captures de cette sous-section illustrent la racine SoftHSM2 dans sa première version (TP Root CA).

4.2 CRL / ARL de la racine HSM

La racine publie une CRL (qui sert ici d'ARL puisqu'elle ne signe que des AC) valable un mois et, pour l'instant, sans certificat révoqué :

4.3 Absence d'export de clé privée

On vérifie qu'aucun fichier de clé privée (.key ou .pem privé) n'a été écrit hors du token : la recherche ne renvoie rien, ce qui confirme que la clé reste confinée dans SoftHSM2.

```

          90:f6:15
      Exponent: 65537 (0x10001)
      X509v3 extensions:
      X509v3 Basic Constraints: critical
      CA:TRUE
      X509v3 Key Usage: critical
      Certificate Sign, CRL Sign
      X509v3 Subject Key Identifier:
      15:0A:CE:F9:98:84:82:63:35:5F:48:B2:69:90:BC:CF:22:06:DA:17
      X509v3 Authority Key Identifier:
      15:0A:CE:F9:98:84:82:63:35:5F:48:B2:69:90:BC:CF:22:06:DA:17
      Signature Algorithm: sha256WithRSAEncryption
      Signature Value:

```

Figure 6: Extensions de la racine : Basic Constraints CA:TRUE (critique), Key Usage : Certificate Sign, CRL Sign.

```

(kali@kali)-[~/RootCA]
$ openssl crl -in crl/rootCA.crl -text -noout
Certificate Revocation List (CRL):
  Version 2 (0x1)
  Signature Algorithm: sha256WithRSAEncryption
  Issuer: C=FR, O=TP Crypto, CN=TP Root CA
  Last Update: Jun 23 08:10:42 2026 GMT
  Next Update: Jul 23 08:10:42 2026 GMT
  CRL extensions:
    X509v3 CRL Number:
      4096
  No Revoked Certificates.
  Signature Algorithm: sha256WithRSAEncryption
  Signature Value:

```

Figure 7: CRL de la racine SoftHSM2 : Last/Next Update espacés d'un mois, No Revoked Certificates.

```

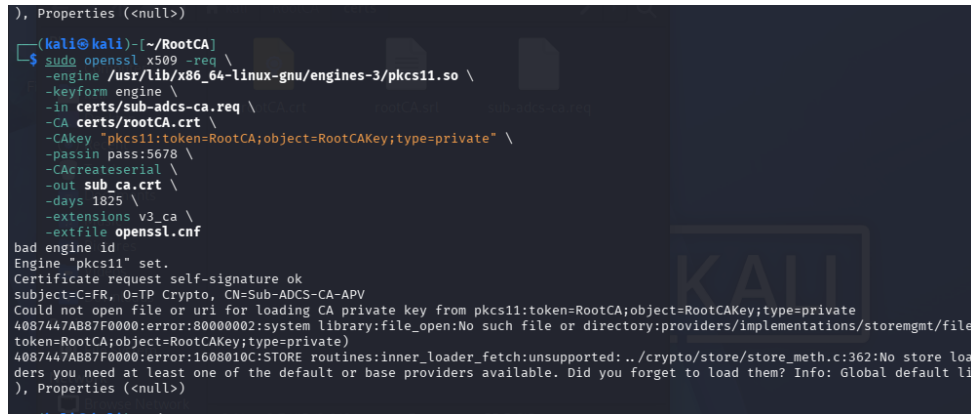
(kali@kali)-[~/RootCA]
$ find . -type f | grep -E "\.key|\.pem"
(kali@kali)-[~/RootCA]

```

Figure 8: `find . -type f | grep -E "\.key|\.pem"` ne retourne aucun fichier : aucune clé privée exportée hors du HSM.

4.4 Point technique PKCS#11

Le moteur (*engine*) historique `pkcs11` est bien listé comme disponible par OpenSSL, mais l'ancienne syntaxe `openssl x509 -req -engine pkcs11` échoue sous OpenSSL 3 (« `bad engine id` », clé privée PKCS#11 introuvable). La solution retenue est le *provider* OpenSSL 3 `pkcs11prov`, utilisé pour toutes les opérations de signature ci-dessus et ci-dessous.



```

), Properties (<null>)
(kali@kali)-[~/RootCA]
$ sudo openssl x509 -req \
-engine /usr/lib/x86_64-linux-gnu/engines-3/pkcs11.so \
-keyform engine \
-in certs/sub-adcs-ca.req \
-CA certs/rootCA.crt \
-CAkey "pkcs11:token=RootCA;object=RootCAKey;type=private" \
-passin pass:5678 \
-CAcreateserial \
-out sub_ca.crt \
-days 1825 \
-extensions v3_ca \
-extfile openssl.cnf
bad engine id
Engine "pkcs11" set.
Certificate request self-signature ok
subject=C=FR, O=TP Crypto, CN=Sub-ADCS-CA-APV
Could not open file or uri for loading CA private key from pkcs11:token=RootCA;object=RootCAKey;type=private
4087447AB87F0000:error:80000002:system library:file_open:No such file or directory:providers/implementations/storemgmt/file
token=RootCA;object=RootCAKey;type=private)
4087447AB87F0000:error:1608010c:STORE routines:inner_loader_fetch:unsupported:../crypto/store/store_meth.c:362:No store load
ders you need at least one of the default or base providers available. Did you forget to load them? Info: Global default li
), Properties (<null>)
(kali@kali)-[~/RootCA]

```

Figure 9: Échec de la voie *engine* (`-engine pkcs11`) : « `bad engine id` » et impossibilité de charger la clé privée PKCS#11. La signature passera par le *provider* `pkcs11prov`.

4.5 Autorité subordonnée ADCS Windows

4.5.1 Objectif et vocabulaire

La dernière étape consiste à faire d'un **Windows Server 2019** une AC subordonnée de notre racine HSM Linux. Autrement dit, la clé privée de cette sous-AC est générée et conservée sur Windows, mais son certificat est signé par la racine `HSM_Root_CA_TRI` restée côté Linux/SoftHSM2. On obtient une chaîne hétérogène Linux → Windows, ce qui illustre l'interopérabilité du standard X.509.

Quelques termes Microsoft à connaître :

- **ADCS** (*Active Directory Certificate Services*) : le rôle Windows qui fournit une autorité de certification. Le service système correspondant s'appelle `CertSvc`.
- **Standalone** (autonome) vs **Enterprise** : une AC *Enterprise* s'intègre à un annuaire Active Directory (gabarits, auto-enrôlement). Une AC *Standalone* est indépendante de tout domaine ; c'est ce que demande le TP, car la VM n'est pas contrôleur de domaine.
- **Subordinate** (subordonnée) vs **Root** : une AC *Root* s'auto-signe ; une AC *Subordinate* génère une *CSR* (PKCS#10) qu'une autorité parente doit signer. Ici l'autorité parente est notre racine SoftHSM2 Linux.

Note

Nom de l'AC. Le nom logique du TP est « `APV` », mais ADCS gère mal les soulignés (`_`) dans le *Common Name* d'une AC (problèmes d'encodage ASN.1/Unicode). On a donc normalisé le nom Windows en `Sub-ADCS-CA-APV` (avec des tirets). C'est purement cosmétique et n'affecte pas la cryptographie.

Toute la procédure est automatisée par le script `windows-adcs/run-on-windows.ps1` les commandes ci-dessous sont celles qu'il exécute, détaillées pour le compte rendu. Le déroulement est en **deux passages** sur Windows, séparés par une signature côté Linux.

4.5.2 Premier passage Windows : rôle ADCS et génération de la CSR

On copie le dossier windows-adcs sur le Bureau de l'administrateur, puis dans une console **PowerShell** ouverte en administrateur :

```
cd C:\Users\Administrateur\Desktop\windows-adcs
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
.\run-on-windows.ps1          # ajouter -ResetADCS si une VM a déjà servi
```

Ce passage exécute pour l'essentiel :

```
# 1) Installer le rôle d'autorité de certification
Install-WindowsFeature -Name ADCS-Cert-Authority -IncludeManagementTools

# 2) Créer une AC subordonnée AUTONOME : cle RSA 4096 + SHA-256,
#    et produire la CSR à faire signer par la racine Linux.
Install-AdcsCertificationAuthority '
  -CAType StandaloneSubordinateCA '
  -CACommonName "Sub-ADCS-CA-APV" '
  -CADistinguishedNameSuffix "O=TP Crypto, C=FR" '
  -CryptoProviderName "RSA#Microsoft Software Key Storage Provider" '
  -KeyLength 4096 '
  -HashAlgorithmName SHA256 '
  -OutputCertRequestFile C:\TP-Crypto-ADCS\output\sub-adcs-ca.req '
  -IgnoreUnicode -Force
```

Note

On ne précise volontairement *pas* `-ValidityPeriod` ici : pour une AC *subordonnée*, la durée de validité est imposée par l'autorité parente au moment où elle signe la CSR (côté Linux, on signera pour 5 ans). C'est normal que le service `CertSvc` ne démarre pas encore : il manque le certificat signé.

Réponse

Le rôle *Autorité de certification* (ADCS-Cert-Authority) est bien installé (figure 10) et le fichier `C:\TP-Crypto-ADCS\output\sub-adcs-ca.req` (requête PKCS#10, figure 11) est créé. Le script invite alors à le faire signer côté Linux.

```
PS C:\Users\Administrateur> Get-WindowsFeature ADCS*

Display Name                                     Name                                     Install State
-----
[X] Autorité de certification                    ADCS-Cert-Authority                    Installed
[ ] Inscription de l'autorité de certification v... ADCS-Web-Enrollment                    Available
[ ] Répondeur en ligne                          ADCS-Online-Cert                      Available
[ ] Service d'inscription de périphérique réseau ADCS-Device-Enrollment                Available
[ ] Service Web Inscription de certificats       ADCS-Enroll-Web-Svc                   Available
[ ] Service Web Stratégie d'inscription de certi... ADCS-Enroll-Web-Pol                   Available

PS C:\Users\Administrateur> _
```

Figure 10: `Get-WindowsFeature ADCS*` : le rôle *Autorité de certification* (ADCS-Cert-Authority) est à l'état `Installed`.

4.5.3 Signature de la CSR côté Linux (racine SoftHSM2)

On rapatrie `sub-adcs-ca.req` sur le poste Linux, puis on la signe avec la clé de la racine restée dans le HSM, toujours via le provider `pkcs11prov` (la clé privée n'est jamais exportée) :

```
-----BEGIN NEW CERTIFICATE REQUEST-----
MIIFGzCCAQMCAQAwOzELMAkGA1UEBhMCRIIEjAQBGNVBAoTCVRQIENyeXB0bzEY
MBYGA1UEAxMPU3ViLUFEQ1MtQ0EtQVBWMIICjANBgkqhkiG9w0BAQEFAAOCAg8A
MIICCGKCAgEAtBtb5djbY2Y1n9yEgGuC2wFDWgpr9adnVy7JNC8Rw838bZ0+g1R
EPKjhEPgRo9VoyV8R3R6/zo1pVMfJXw68yCfCq50Th/G9Fm2femNK8WGEQ9N44jh
k7krcFSk59eLzhbec8ciQvGkw3JiHULbC2JsyvvY9K36G2Jyom93YcsfdjaPrBbB
XYoumHuCM2Cvv0F19XVn5J+f1Ijhzk9iBnVPw42UY93hYY89hFV0MeItL8XXu1T
PVYt4pH7hd90jCKiOfVGEZMpFRSCMdL6Y481HnFApxsP18cWrxbaL00qZQeKCuxa
6/DQ4R0Do8c9s12sCQTb3TSH+8Q4yHFuJ5kFbK114xUJTs3dF7/RKUax7kwXQ1k
AqWTP8gaqPyJcathh0RK9Ip0yywqB8vcthH9prbKioIpD0EvJQYNAicJ21hhL+21
1iAwanvrt1ZPQ6LinjKwXdcBbM910XmMRM2nmD6HyyXEhK5Qplwly0rrefb0Fp/QC
b9mCtXm9y60a1ofg7CpujwIoXGnPcboxC/hjA1J7qJucnp0qF9dkovQI17Atnm+n
5MNe8jDY3KZ2XTb93b8dG0+DuaV8ioAyt/3Iy0s8Dt++aCwqFLZW2mF/4Pouc1i
6T8f+zIV7uavsYEyh+N9NFP0FrFdcX8fh6H+IVjKqCwqP7WDM9zLJkCAwEAACB
mjadBgorBgEEAYI3DQIDMQ8WDEwLjAuMjA5NDguMi4weQYJKoZIhvcNAQkOMWww
ajAQBgkrBgEEAYI3FQEEAwIBADAdBgNVHQ4EFgQUUpnSwYdSyr2kjs/7RAhc1u42
JdYwQYJKwYBBAGCNxQCBAweCgBTAHUAYgBDAEEwCwYDVR0PBAQDAgGMA8GA1Ud
EwEB/wQFMAMBAf8wDQYJKoZIhvcNAQELBQADggIBAAR/HHG91Jn1S37Amr978mDi
013IhpQ+2xz9PJ009S+mXZs1es9RMwWkZgXrT/BdBwf+bH+Fdw9Djms8SyevrK
8wkRJaQpZSIRQZ7BmLUHCQq9Gxdp5xe6z1IGIhpTiKPH16fHMjtT9rBWExgOqBfn
ZN+/dRfMnxH241Br0v3YX4oBw1u2IGMB11atXd4MydPKo9QgD7xNC4DzKX1pBCG
+AhmmDUYZNSfjPXDRHrSvAMwiHPwSBk+gsxLZ2k5SajQf6cjB1W82sj0R4JYInjF
2MwZMsT9iOafzhxUthBcXIzvnIP+20esMOUTwTm9nHiY/sEOKgioPHYfmyKXk+rr
9NOpQZPEYBNb9CZJhUak6LnJX5AybD2rPQFkjy4Eig09i1A0BK6hVurzRr22f98Q
mtOuhZfueY1b9ZPJ6LiWkjhOZWGFDb2d34kH7qrx9R3kaycIEjhJmxD4KHHqJA6m
eWDRgB7BzCqyoYfQCE/2a60xmtuu5bV2ABHtOdJ/C3FzVVGDEnWRQGT9edS3tWM
XdaE/TEOY6+5mMy2nYAWgwK1k352h6oNYHVXddttMf1bYsuAFWJwOZHeAw9rAQGG
8D6r+vsEwvjKMBRLD8mv8ZMFbe8ICM8Kyo8uEJN9DRqD19VnHtYmP1ogg6bXb
hw7nGk8nVohwZQ4yGMSw
-----END NEW CERTIFICATE REQUEST-----
```

Figure 11: La requête PKCS#10 de l'AC subordonnée (---BEGIN NEW CERTIFICATE REQUEST---) générée par ADCS, à transmettre à la racine Linux.

```
openssl x509 -req -provider default -provider pkcs11prov \
-in certs/sub-adcs-ca.req \
-CA certs/rootCA.crt \
-CAkey "pkcs11:token=RootCA;object=RootCAKey;type=private;pin-value=5678" \
-CAcreateserial -days 1825 \
-extensions v3_ca -extfile openssl.cnf \
-out sub_ca.crt
```

Le profil d'extensions appliqué (v3_ca) fait bien de la cible une AC subordonnée :

```
[v3_ca]
basicConstraints      = critical, CA:true
keyUsage              = critical, digitalSignature, cRLSign, keyCertSign
subjectKeyIdentifier = hash
authorityKeyIdentifier = keyid:always,issuer
```

Note

-days 1825 fixe la validité de l'AC subordonnée à 5 ans : c'est bien la racine parente qui décide de la durée, comme annoncé au premier passage Windows.

Réponse

Le certificat `sub_ca.crt` obtenu est émis par `CN=RootCA` pour le sujet `CN=Sub-ADCS-CA-APV, O=TP Crypto, C=FR`, valable du 23/06/2026 au 22/06/2031 (5 ans) — voir la vérification figure 14.

4.5.4 Second passage Windows : import de la racine et finalisation

On recopie côté Windows le certificat signé `sub_ca.crt` et le certificat racine `rootCA.crt`, puis on importe la racine comme ancre de confiance et on installe le certificat de la sous-AC :

```
# Importer la racine Linux comme ancre de confiance (cf. tâche 15)
```

```
certutil -addstore Root C:\TP-Crypto-ADCS\input\rootCA.crt
# Installer le certificat signe dans l'AC subordonnee, puis demarrer le service
certutil -installCert C:\TP-Crypto-ADCS\input\sub_ca.crt
Start-Service CertSvc
```

La racine SoftHSM2 est alors bien présente dans le magasin *Autorités de certification racines de confiance* de la machine (figure 12).

```
PS C:\Users\Administrateur> certutil -store Root "RootCA"
Root "Autorités de certification racines de confiance"
===== Certificat 2 =====
Numéro de série : 705dbffcd415bcffcb53179394a8a0d7e044b93d
Émetteur: CN=RootCA, O=TP Crypto, C=FR
  NotBefore : 23/06/2026 14:23
  NotAfter : 20/06/2036 14:23
Objet: CN=RootCA, O=TP Crypto, C=FR
La signature correspond à la clé publique
Certificat racine : le sujet correspond à l'émetteur
Hach. cert. (sha1) : a46199b09c0967877e143643e01af764c5c4d3f2
Aucune information sur le fournisseur de clé
Impossible de trouver le certificat et la clé privée pour le déchiffrement
CertUtil: -store La commande s'est terminée correctement.
PS C:\Users\Administrateur>
```

Figure 12: `certutil -store Root "RootCA"` : la racine SoftHSM2 (CN=RootCA, O=TP Crypto) est installée comme ancre de confiance, « le sujet correspond à l'émetteur » (racine auto-signée).

4.5.5 Vérification de la chaîne hétérogène Linux → Windows

C'est le résultat central de la Partie 3 : le certificat de l'AC subordonnée, hébergée sous Windows, se valide jusqu'à la racine SoftHSM2 importée.

```
certutil -verify -urlfetch .\sub_ca.crt
```

Réponse

La chaîne Sub-ADCS-CA-APV → RootCA est validée par Windows (« Ce certificat est valide », figure 13). Windows fait donc confiance à une AC dont l'ancre est une racine dont la **clé privée n'a jamais quitté le HSM SoftHSM2 sous Linux**. L'objectif d'interopérabilité Linux/Windows autour d'une racine adossée à un HSM est atteint. Le seul avertissement de `certutil -verify` porte sur l'absence de point de révocation en ligne, attendu pour une PKI de laboratoire hors ligne.

4.5.6 Émission d'un certificat TLS par l'AC ADCS

Une fois l'AC subordonnée opérationnelle (CertSvc en Running), elle peut émettre des certificats d'entité finale. On génère une CSR TLS à partir d'un fichier `.inf`, puis on la soumet à l'AC :

```
# CSR TLS depuis le modele .inf (RSA 3072, EKU serverAuth, SAN dns=www.tp-crypto.local)
certreq -new C:\TP-Crypto-ADCS\output\web-tls.inf C:\TP-Crypto-ADCS\output\web-tls.req
certreq -submit -config "$env:COMPUTERNAME\Sub-ADCS-CA-APV" '
C:\TP-Crypto-ADCS\output\web-tls.req C:\TP-Crypto-ADCS\output\web-tls.cer
```

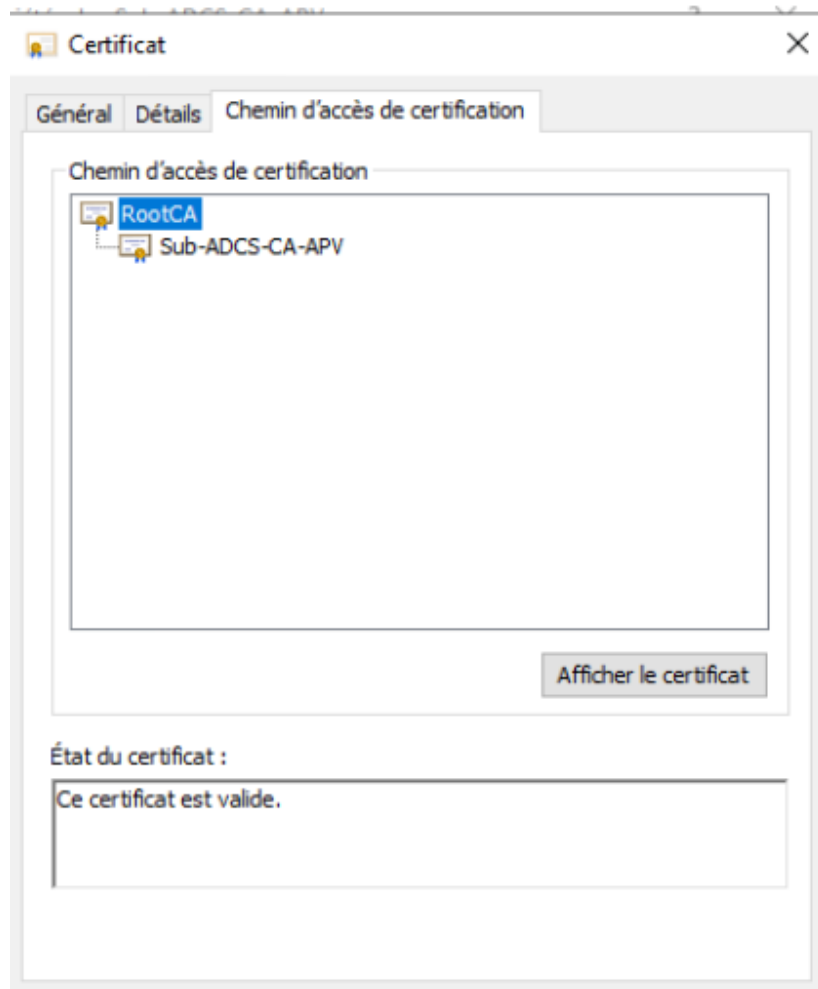


Figure 13: Onglet *Chemin d'accès de certification* : la chaîne `RootCA` $\rightarrow$ `Sub-ADCS-CA-APV` est complète et « Ce certificat est valide ».

```

CertUtil: -viewstore La commande s'est terminée correctement.
PS C:\Users\Administrateur> certutil -verify -urlfetch .\sub_ca.crt
Emetteur:
  CN=RootCA
  O=TP Crypto
  C=FR
Hachage du nom (sha1) : 910e944076e43f58f8704610d3296309bb2dfa3f
Hachage du nom (md5) : aa5154f664b1638c143ed7aeb5b784b8
Objet:
  CN=Sub-ADCS-CA-APV
  O=TP Crypto
  C=FR
Hachage du nom (sha1) : 6f2a0abde244e16c2e0a2e94846b4ecff135ba8a
Hachage du nom (md5) : cf5a913d91a72e2377a25ce8611c6e90
Numéro de série du certificat : 29b147ba4655e5a4aa03a934a2205fe47e583add

dwFlags = CA_VERIFY_FLAGS_CONSOLE_TRACE (0x20000000)
dwFlags = CA_VERIFY_FLAGS_DUMP_CHAIN (0x40000000)
ChainFlags = CERT_CHAIN_REVOCATION_CHECK_CHAIN_EXCLUDE_ROOT (0x40000000)
HCCE_LOCAL_MACHINE
  Pas d'URL "Aucun" Heure : 0 (null)
  ----- Protocole OCSP du certificat -----
  Pas d'URL "Aucun" Heure : 0 (null)
  -----

Exclude leaf cert:
  Chain: ball1d4203b49c06003b654a836b4e86d9fe19c49
Full chain:
  Chain: 495bc9b5970c26874714bfbf83c30c674a131e26
  Issuer: CN=RootCA, O=TP Crypto, C=FR
  NotBefore: 23/06/2026 14:23
  NotAfter: 22/06/2031 14:23
  Subject: CN=Sub-ADCS-CA-APV, O=TP Crypto, C=FR
  Serial: 29b147ba4655e5a4aa03a934a2205fe47e583add
  Cert: ball1d4203b49c06003b654a836b4e86d9fe19c49
La fonction de révocation n'a pas pu vérifier la révocation du certificat. 0x80092012 (-2146885614 CRYPT_E_NO_REVOCATION_CHECK)
-----
Vérification de révocation ignorée -- aucune information concernant la
révocation n'est disponible
Cert est un certificat d'autorité de certification
Impossible de vérifier l'état de révocation du certificat feuille
CertUtil: -verify La commande s'est terminée correctement.
PS C:\Users\Administrateur>

```

Figure 14: `certutil -verify` sur `sub_ca.crt` : émetteur `CN=RootCA`, sujet `CN=Sub-ADCS-CA-APV`, validité 23/06/2026 → 22/06/2031, chaîne complète reconstruite jusqu'à la racine. La seule remarque (`CRYPT_E_NO_REVOCATION_CHECK`) est normale : aucune URL de CRL/OCSP en ligne pour cette racine de TP.

Note

Une AC *Standalone* met par défaut les demandes *en attente* : le script les valide automatiquement (`certutil -resubmit` puis `certreq -retrieve`). Le certificat émis porte alors l'EKU *TLS Web Server Authentication* et le SAN `DNS:www.tp-crypto.local`, et se vérifie avec `certutil -verify web-tls.cer`. Tous les artefacts et journaux sont regroupés dans `C:\TP-Crypto-ADCS\` (rapport `RAPPORT-WINDOWS-ADCS.md` et dossier `logs\`).